Arquitectura de Computadores

Practica 1

Jerarquia de Memoria y Comportamiento de Cache

Estudio del efecto de la localidad de los accesos a memoria en las prestaciones de programas en microprocesadores

---

Plataforma:	FinisTerrae III (CESGA)
Procesador:	Intel Xeon Platinum 8352Y (Ice Lake-SP)
Compilador:	`gcc -O0`
Jobs Slurm:	5366644, 5367628, 5367629, 5367630
Strides D:	1, 8, 16, 64, 128, 256, 512 (7 valores)
Tamanos L:	11 valores, de 384 a 3 145 728 lineas
Mediciones:	$3 \times 7 \times 11 \times 10 = 2\,310$ totales
Metrica:	Media geometrica de los 3 mejores de 10 repeticiones

February 26, 2026

Contents

1 Introduccion y objetivos **2**

2 Arquitectura del sistema **2**
2.1 Jerarquia de memoria . . . 2
2.2 Prefetcher hardware (microarquitectura Sunny Cove) . . . 2

3 Metodologia **3**
3.1 Parametros del experimento . . . 3
3.2 Calculo de R . . . 3
3.3 Configuracion Slurm: nodo dedicado con `-c 64` . . . 3
3.4 Tratamiento estadistico . . . 4

4 Implementacion **4**
4.1 `acp1.c` — experimento base . . . 4
4.2 `acp1_directo.c` y `acp1_int.c` . . . 4

5 Resultados experimentales **5**
5.1 Double + acceso indirecto (`acp1.c`) . . . 5
5.2 Int + acceso indirecto (`acp1_int.c`) . . . 5
5.3 Double + acceso directo (`acp1_directo.c`) . . . 6

6 Graficas de resultados **7**
6.1 Experimento base con 7 strides . . . 7
6.2 Comparativa global de los tres experimentos . . . 8
6.3 Comparativa por stride extremo: D=1 y D=16 . . . 10

7 Analisis por zonas de la jerarquia **11**
7.1 Zona L1 — $L \leq 768$ lineas (footprint $\leq$ 48 KB) . . . 11
7.2 Zona L2 — $768 < L \leq 20\,480$ lineas (footprint 72 KB–960 KB) . . . 11
7.3 Zona L3 — $20\,480 < L \leq 786\,432$ lineas (footprint 2.5 MB–48 MB) . . . 11
7.4 Zona RAM — $L > 786\,432$ lineas (footprint > 48 MB) . . . 11

8 La escalera completa de la jerarquia **12**

9 Analisis comparativo de variantes **12**
9.1 Double vs. Int . . . 12
9.2 Directo vs. Indirecto . . . 13

10 Latencias identificadas experimentalmente **13**

11 Conclusiones **13**

A Configuracion Slurm **14**

1 Introduccion y objetivos

Esta practica caracteriza experimentalmente la latencia de cada nivel de la jerarquia de memoria del procesador Intel Xeon Platinum 8352Y mediante una operacion de reduccion de suma sobre un vector con patron de acceso controlado. Se varian dos parametros:

- **D (stride):** separacion en elementos entre posiciones accedidas. Controla la localidad espacial y la eficacia del hardware prefetcher.
- **L (lineas):** numero de lineas de cache distintas referenciadas. Determina en que nivel de la jerarquia residen los datos durante el experimento.

El experimento base accede de forma **indirecta** mediante el vector de indices `ind[]`. Se realizan dos experimentos adicionales: con tipo `int` (4 bytes en lugar de 8) y con acceso **directo** eliminando `ind[]`.

2 Arquitectura del sistema

2.1 Jerarquia de memoria

Table 1: Jerarquia de cache del Intel Xeon Platinum 8352Y (Ice Lake-SP)

Nivel	Capacidad	Asoc.	S (lineas)	Latencia aprox.
L1d	48 KB	12-way	$S_1 = 768$	∼5 ciclos
L2	1.25 MB	20-way	$S_2 = 20\,480$	∼14 ciclos
L3	48 MB	12-way	$S_3 = 786\,432$	∼40–50 ciclos
RAM	256 GB DDR4	—	—	∼150–200 ciclos

Tamano de linea de cache: **CLS = 64 bytes**. Los experimentos se ejecutan en **un unico core** con el nodo completo reservado (`-c 64`).

2.2 Prefetcher hardware (microarquitectura Sunny Cove)

El procesador implementa cuatro prefetchers hardware independientes:

DCU Streamer: Precarga la siguiente linea de cache ante accesos ascendentes. Se activa con cualquier patron secuencial.

DCU IP Stride Prefetcher: Rastrea instrucciones de carga individuales. Cuando detecta un stride regular, emite precargas anticipadas hasta 2 KB de distancia.

L2 Streamer: Monitoriza peticiones desde L1 y corre hasta **20 lineas por delante**. Gestiona hasta 32 streams. No cruza limites de pagina de 4 KB.

L2 Spatial (Adjacent Cache Line): Completa cada linea cargada en L2 con su linea adyacente (bloques de 128 bytes). Su actuacion independiente del patron provoca *prefetch pollution* con $D = 16$.

3 Metodologia

3.1 Parametros del experimento

Strides D

Se usan **7 valores** potencia de 2: $D \in \{1, 8, 16, 64, 128, 256, 512\}$. Los dos valores nuevos son $D = 256$ (stride de 16 KB) y $D = 512$ (stride de 32 KB), que permiten observar el comportamiento cuando el L2 Streamer es completamente ineficaz y la latencia se estabiliza en su valor maximo.

Valores de L

Table 2: Valores de L y nivel de jerarquia correspondiente

L (lineas)	Fraccion	Footprint	Nivel
384	$0.5 \times S_1$	24 KB	L1 (holgado)
1 152	$1.5 \times S_1$	72 KB	L2 (desborda L1)
10 240	$0.5 \times S_2$	640 KB	L2 (holgado)
15 360	$0.75 \times S_2$	960 KB	L2 (ajustado)
40 960	$2 \times S_2$	2.5 MB	L3 (desborda L2)
81 920	$4 \times S_2$	5 MB	L3
163 840	$8 \times S_2$	10 MB	L3
524 288	$0.67 \times S_3$	32 MB	L3 (llenando)
786 432	$1 \times S_3$	48 MB	Limite exacto L3
1 572 864	$2 \times S_3$	96 MB	**RAM real**
3 145 728	$4 \times S_3$	192 MB	**RAM profunda**

3.2 Calculo de R

$$R = \frac{L \times \text{CLS}}{D \times \text{sizeof(tipo)}}, \qquad N = (R - 1) \cdot D + 1$$

3.3 Configuracion Slurm: nodo dedicado con `-c 64`

Los tres scripts de Slurm se configuran con `#SBATCH -c 64`, reservando los **64 cores logicos del nodo completo** aunque solo se utilice uno para la medicion. Esta decision es crucial por cuatro razones:

1. Garantiza que ningun otro proceso del sistema de colas comparte el nodo, eliminando contaminacion por trafico de cache externo.
2. Asegura que L1 y L2 (privadas por core) son exclusivas del proceso de medicion.
3. Evita que el planificador del SO migre el proceso a otro core con cachés en estado diferente.

4. Reduce la variabilidad en la zona L3 (compartida entre los 36 cores fisicos del socket), donde otros procesos provocarian expulsiones de lineas ajenas.

3.4 Tratamiento estadistico

Para cada combinacion (D, L): 10 repeticiones externas, media geometrica de los 3 mejores valores de ciclos/acceso.

4 Implementacion

4.1 acp1.c — experimento base

```c
// R = lineas * bytes/linea / (stride * bytes/elem)
long long R = (long long)L * CLS / (D * sizeof(double));
long long N = (R - 1) * D + 1;

// Reserva alineada: inicio de A[] coincide con inicio de linea de cache
double *A   = (double*)aligned_alloc(CLS, N * sizeof(double));
int    *ind = (int*)malloc(R * sizeof(int));

// Inicializacion = calentamiento de cache y TLBs
for (long long i = 0; i < R; i++) ind[i] = i * D;
for (long long i = 0; i < R; i++) A[ind[i]] = (double)rand() / RAND_MAX;

// Medicion de ciclos
start_counter();
for (int k = 0; k < REPS; k++) {
    double suma = 0.0;
    for (long long i = 0; i < R; i++)
        suma += A[ind[i]];   // Acceso indirecto: ind[i] -> A[]
    acum[k] = suma;
}
double ciclos_totales = get_counter();
double ciclos_por_acceso = ciclos_totales / ((double)R * REPS);
```

Listing 1: Calculo de R y bucle de medicion (acp1.c)

4.2 acp1_directo.c y acp1_int.c

```c
// --- acp1_directo.c: acceso directo sin ind[] ---
for (long long i = 0; i < R; i++)
    suma += A[i * D];  // Un acceso a memoria (vs. dos en el caso indirecto)

// --- acp1_int.c: tipo int (4 bytes, R = el doble) ---
long long R = (long long)L * CLS / (D * sizeof(int)); // R * 2
int *A = (int*)aligned_alloc(CLS, N * sizeof(int));
int suma = 0;
for (long long i = 0; i < R; i++)
    suma += A[ind[i]]; // ADD entero: menor latencia que FADD flotante
```

Listing 2: Diferencias clave de las variantes adicionales

5 Resultados experimentales

Codigo de colores en las tablas: verde < 8 ciclos / naranja 8–12 ciclos / rojo > 12 ciclos.

5.1 Double + acceso indirecto (`acp1.c`)

Table 3: Ciclos/acceso — Double indirecto. 7 strides × 11 valores de L.

L	Nivel	D=1	D=8	D=16	D=64	D=128	D=256	D=512
384	L1	7.08	7.13	7.07	7.07	7.14	7.65	8.77
1 152	L2	7.09	7.16	7.19	7.17	7.23	7.30	7.29
10 240	L2	7.12	7.18	7.20	7.20	7.29	7.26	7.35
15 360	L2	7.12	7.22	7.21	7.28	7.41	7.30	7.43
40 960	L3	7.12	7.35	7.65	7.81	7.83	7.48	7.97
81 920	L3	7.12	7.28	7.63	7.63	7.48	7.49	7.63
163 840	L3	7.13	7.29	7.64	7.56	7.48	7.54	7.69
524 288	L3	7.18	8.85	13.00	13.85	13.65	14.06	13.07
786 432	L3 lim.	7.19	9.48	14.80	14.93	14.35	15.07	15.05
1 572 864	**RAM**	7.20	9.81	16.18	16.61	15.30	15.70	17.26
3 145 728	**RAM**	7.20	9.85	17.13	17.70	15.81	16.29	17.16

5.2 Int + acceso indirecto (`acp1_int.c`)

Table 4: Ciclos/acceso — Int indirecto.

L	Nivel	D=1	D=8	D=16	D=64	D=128	D=256	D=512
384	L1	6.76	6.80	6.83	6.80	6.85	6.92	7.41
1 152	L2	6.77	6.82	7.17	7.24	7.15	7.24	7.42
10 240	L2	6.79	6.83	7.18	7.22	7.27	7.33	7.20
15 360	L2	6.79	6.86	7.22	7.22	7.29	7.40	7.17
40 960	L3	6.80	6.90	7.31	7.81	7.49	7.48	7.52
81 920	L3	6.79	6.88	7.30	7.69	7.54	7.43	7.46
163 840	L3	6.79	6.88	7.29	7.77	7.60	7.46	7.47
524 288	L3	6.80	7.46	8.65	14.44	13.81	13.54	13.91
786 432	L3 lim.	6.80	7.52	9.25	15.83	14.72	14.50	14.82
1 572 864	**RAM**	6.81	7.56	9.64	17.41	16.60	15.35	15.73
3 145 728	**RAM**	6.81	7.57	9.71	18.40	17.61	15.87	16.19

5.3 Double + acceso directo (`acp1_directo.c`)

Table 5: Ciclos/acceso — Double directo.

L	Nivel	D=1	D=8	D=16	D=64	D=128	D=256	D=512
384	L1	7.06	6.97	6.82	6.30	6.46	7.09	8.13
1 152	L2	7.08	7.09	7.06	6.86	6.80	6.95	7.28
10 240	L2	7.10	7.16	7.13	7.11	7.07	7.03	6.99
15 360	L2	7.10	7.21	7.23	7.17	7.14	7.21	7.20
40 960	L3	7.12	7.36	7.52	7.45	7.47	7.64	7.68
81 920	L3	7.11	7.29	7.45	7.28	7.21	7.25	7.32
163 840	L3	7.11	7.30	7.43	7.30	7.23	7.25	7.33
524 288	L3	7.14	8.44	12.09	12.50	12.38	12.90	12.22
786 432	L3 lim.	7.15	8.86	13.60	13.65	13.23	13.83	14.17
1 572 864	**RAM**	7.16	9.24	15.17	15.52	14.59	15.10	15.60
3 145 728	**RAM**	7.16	9.26	16.03	16.29	15.17	15.85	16.27

6 Graficas de resultados

6.1 Experimento base con 7 strides

grafica_memoria.png

Figure 1: Ciclos/acceso vs. lineas L (escala log) para el experimento base (double + indirecto, 7 strides). Las lineas verticales marcan los limites de L1 ($S_1 = 768$, rojo) y L2 ($S_2 = 20\,480$, verde). Se aprecian claramente tres regiones: zona plana L1/L2 (prefetcher eficaz), escalada L3 y estabilizacion RAM.

6.2 Comparativa global de los tres experimentos

comparativa_experimentos.png

Figure 2: Comparativa de los tres experimentos en tres paneles paralelos. Cada panel muestra los 7 strides. Las lineas verticales marcan S_1 (rojo), S_2 (verde) y S_3 (naranja). A partir de $L = 524\,288$ los datos desbordan L3 y la latencia escala abruptamente para $D \geq 16$.

6.3 Comparativa por stride extremo: D=1 y D=16

comparativa_D1.png

(a) Stride $D = 1$: las tres variantes permanecen completamente planas en toda la jerarquia, incluyendo RAM con 192 MB de footprint. El hardware prefetcher oculta por completo todas las latencias. La variante int es $\sim$0.3 ciclos mas rapida por la menor latencia de la suma entera.

comparativa_D16.png

7 Analisis por zonas de la jerarquia

7.1 Zona L1 — $L \leq 768$ lineas (footprint $\leq$ 48 KB)

Con 24 KB de footprint, los datos caben completamente en la L1d de 48 KB. Las curvas convergen en el rango 6.3–7.7 ciclos en todos los strides y variantes.

El unico comportamiento anomalo es $D = 512$ en $L = 384$: con tan pocos elementos ($R = 6$ para double), el overhead fijo del bucle (control de iteracion, calculo de multiplicacion $i \times D$) domina sobre el coste del acceso real, inflando artificialmente el valor a 8.77 ciclos. Esto no refleja la latencia de L1 sino el coste de computacion del bucle sin optimizacion.

La ventaja de `int` ($\sim$0.3 ciclos menos que double) se debe a que la instruccion de suma entera `ADD` tiene menor latencia que `FADD` de punto flotante. La ventaja del acceso directo con $D = 64$ (6.30 vs. 7.07 ciclos) refleja la eliminacion de la lectura de `ind[]`.

7.2 Zona L2 — $768 < L \leq 20\,480$ lineas (footprint 72 KB–960 KB)

A pesar de desbordar L1 (latencia teorica de L2 $\sim$14 ciclos), los valores medidos se mantienen por debajo de 7.5 ciclos en todos los casos. El **DCU IP Stride Prefetcher** detecta el stride constante y emite precargas desde L2 hacia L1 antes de que sean necesarias, ocultando virtualmente toda la latencia de L2.

La penalizacion al cruzar de L1 a L2 es inferior a 0.2 ciclos, confirmando la eficacia del prefetcher incluso para strides de hasta $D = 512$.

7.3 Zona L3 — $20\,480 < L \leq 786\,432$ lineas (footprint 2.5 MB–48 MB)

Se distinguen dos subzonas. En la **L3 baja** ($L = 40\,960$–$163\,840$, footprint 2.5–10 MB), todos los strides se mantienen por debajo de 8 ciclos: el L2 Streamer sigue siendo eficaz. En la **L3 alta** ($L = 524\,288$–$786\,432$, 32–48 MB) se produce la transicion critica, con las curvas separandose claramente:

$D = 1$**:** Permanece plano en $\sim$7.2 ciclos. El L2 Streamer (20 lineas por delante) oculta la latencia de L3 con la misma eficacia que la de L2.

$D = 8$**:** Sube progresivamente hasta $\sim$9.5 ciclos. El prefetcher sigue activo pero con menor eficacia al crecer la separacion entre accesos consecutivos.

$D = 16$**:** Peor caso ($\sim$14.8 ciclos en el limite L3). El L2 Spatial Prefetcher genera *prefetch pollution*: por cada linea util de 64 bytes trae otra de 64 bytes adyacente innecesaria, duplicando el trafico L2-L3 y degradando el rendimiento.

$D \geq 64$ **hasta** $D = 512$**:** Latencias similares (13–15 ciclos) que D=16. El stride supera ampliamente el umbral del L2 Streamer. Paradojicamente no empeoran respecto a D=16 porque el prefetcher reduce su actividad, eliminando tambien la pollution.

7.4 Zona RAM — $L > 786\,432$ lineas (footprint $>$ 48 MB)

La transicion L3→RAM es el hallazgo mas relevante. Entre $L = 786\,432$ y $L = 1\,572\,864$ se produce un **cambio de pendiente** apreciable para $D \geq 8$. Los valores se estabilizan

entre $L = 1\,572\,864$ y $L = 3\,145\,728$, confirmando que se ha alcanzado la latencia de RAM pura.

Table 6: Ciclos/acceso en RAM profunda ($L = 3\,145\,728$, footprint 192 MB)

Variante	**D=1**	**D=8**	**D=16**	**D=64**	**D=128**	**D=256**	**D=512**
Double indirecto	7.20	9.85	17.13	17.70	15.81	16.29	17.16
Int indirecto	6.81	7.57	9.71	18.40	17.61	15.87	16.19
Double directo	7.16	9.26	16.03	16.29	15.17	15.85	16.27
Mejor	Int	Int	Int	Dir.	Dir.	Int	Int

Observation clave: $D = 1$ es completamente inmune a la jerarquia. Con 192 MB en RAM el coste es ∼7.2 ciclos, identico a L1. Los cuatro prefetchers actuan coordinadamente.

8 La escalera completa de la jerarquia

Con 11 valores de L se puede trazar la escalera completa. Para D=64 (double indirecto):

Table 7: Escalera de latencias para D=64, double indirecto

Zona	**L**	**Footprint**	**Ciclos/acceso**
L1	384	24 KB	7.07
L2	15 360	960 KB	7.28 (+0.2)
L3 entrada	40 960	2.5 MB	7.81 (+0.5)
L3 profunda	786 432	48 MB	14.93 (+7.1)
RAM	3 145 728	192 MB	17.70 (+2.8)

La diferencia L1→RAM es de $\Delta = 10.63$ ciclos, frente a los ∼145 teoricos de RAM pura. El prefetcher atenua la penalizacion en un factor de $\approx$ **14**×.

9 Analisis comparativo de variantes

9.1 Double vs. Int

En la zona L1/L2, `int` es ∼0.3 ciclos mas rapido por la menor latencia de `ADD`. En L3/RAM con strides medios (D=16), la ventaja es mucho mayor: en RAM profunda, `int` obtiene 9.71 ciclos frente a 17.13 del double (mejora del **43%**), porque con el doble de R los elementos por linea son mas utiles y la presion relativa de `ind[]` se reduce.

Sin embargo, para strides grandes en RAM ($D = 64$), `int` es el *peor* con 18.40 ciclos, superando al double indirecto (17.70). La razon: con $R_{\text{int}} = 2 \times R_{\text{double}}$, el vector `ind[]`

tiene el doble de entradas y genera accesos masivos adicionales a RAM para leer los propios indices.

9.2 Directo vs. Indirecto

La eliminacion de `ind[]` beneficia especialmente a strides grandes en L1 (D=64: 6.30 vs. 7.07 ciclos, mejora del 11%) y a la zona RAM con $D = 8$ (9.26 vs. 9.85, mejora del 6%). En RAM con $D \geq 64$ la ventaja se reduce porque el cuello de botella es la latencia de RAM en si misma y no la contention entre streams.

10 Latencias identificadas experimentalmente

Table 8: Latencias teoricas vs. observadas y factor de mejora del prefetcher

Nivel	Lat. teorica	Ciclos observados	Factor mejora
L1d	∼5 ciclos	∼7 ciclos	— (overhead `-O0`)
L2	∼14 ciclos	∼7 ciclos	2× (latencia oculta)
L3	∼45 ciclos	7–15 ciclos	3×–6×
RAM	∼150 ciclos	7–18 ciclos	8×–21×

Los valores en L1 (∼7 ciclos) superan la latencia teorica de 5 ciclos por el overhead del bucle sin optimizacion (`-O0`).

11 Conclusiones

1. **La experimentacion cubre toda la jerarquia.** Con 11 valores de L desde 24 KB hasta 192 MB se caracterizan experimentalmente los cuatro niveles. Los limites $S_1 = 768$, $S_2 = 20\,480$ y $S_3 = 786\,432$ lineas se validan por los cambios de pendiente en las curvas.
2. `-c 64` **es esencial para la fiabilidad.** Reservar el nodo completo elimina la contaminacion por otros procesos y garantiza la exclusividad de L1 y L2, reduciendo la variabilidad de las mediciones.
3. **D=1 es inmune a la jerarquia.** Los cuatro prefetchers actuan coordinadamente ocultando las latencias de L2, L3 y RAM. Con 192 MB en RAM el coste es ∼7.2 ciclos, identico a L1.
4. **La escalera de latencias aparece muy atenuada.** La diferencia L1→RAM para D=64 es de 10.6 ciclos en lugar de los ∼145 teoricos (14× de mejora por el prefetcher).
5. **La transicion L3→RAM es claramente observable** entre $L = 786\,432$ y $L = 1\,572\,864$ para $D \geq 8$, validando experimentalmente $S_3 = 786\,432$ lineas.

6. **D=16 es el peor stride** en double indirecto (17.13 ciclos en RAM) por el fenomeno de *prefetch pollution* del L2 Spatial Prefetcher.
7. **D=256 y D=512 no empeoran significativamente respecto a D=64.** A partir de cierto stride el prefetcher es igualmente ineficaz y la latencia se estabiliza, sin degradacion monotona adicional.
8. **El acceso directo supera al indirecto** en zona L1 con strides grandes y en RAM con strides medios (hasta 11% de mejora).
9. **El tipo `int` supera al `double`** para strides D=8 y D=16 en toda la jerarquia (hasta 43% en RAM). Para D=64 la inversion de resultado evidencia el coste de un `ind[]` de gran tamano.
10. **Los valores se estabilizan en RAM profunda.** La diferencia entre $L = 1\,572\,864$ y $L = 3\,145\,728$ es inferior al 2%, confirmando que el sistema ha alcanzado la latencia de RAM pura.

A Configuracion Slurm

```
#!/bin/bash
#SBATCH -n 1            # 1 proceso (sin MPI)
#SBATCH -c 64           # 64 cores logicos = nodo completo reservado
#SBATCH --mem=4G        # Memoria suficiente para los vectores grandes
#SBATCH -t 00:02:00     # Tiempo estimado de ejecucion
#SBATCH --job-name p1acg_eqNN

# -c 64 reserva el nodo completo aunque solo se use 1 core.
# Esto garantiza exclusividad de L1/L2 (privadas) y evita
# contaminacion de L3 (compartida) por otros procesos.
gcc acp1.c -o acp1 -O0
```

Listing 3: Cabecera de los scripts Slurm (identica en las tres variantes)